Comparaison d'algorithmes de tri en RISC-V

Hubert Lestrat
Polytech-Sorbonne — MAIN3 Architecture des ordinateurs

24 avril 2026

Table des matières

1 Références **1**

2 Astuces **1**

3 Méthode de mesures **1**
3.1 Dans RARS 1

4 Résultats **2**

5 VRAC à travailler **2**
5.1 Temps (= nombre d'instructions) 2
5.2 Mémoire 3

1 Références

- https://www.laurentbloch.net/MySpip3/Mon-premier-programme-en-assembleur-RISC-V
- https://www.laurentbloch.net/MySpip3/Nouvel-episode-de-programmation-en-assembleur-RISC-
- https://www.laurentbloch.net/MySpip3/Assembleur-RISC-V-maintenant-les-tris
- https://fr.wikipedia.org/wiki/Tri_%C3%A0_bulles

2 Astuces

`rars test.asm` ⇒ compile et lance le programme en console.

Attention, lorsqu'on « include » un fichier, les étiquettes doivent rester uniques. Le mieux est de les préfixer avec le nom de la fonction.

Générer un tableau aléatoire de 20 entrées (pour `tab.asm`) :

```
python3 -c "import random; random.seed(); \
  print(', '.join(str(random.randint(-99, 99)) for _ in range(20)))"
```

3 Méthode de mesures

3.1 Dans RARS

- Ouvrir le fichier `.asm` à tester.

- `Tools` → `Instruction Counter`, puis « Connect to program ».
- `Settings` → `Hexadecimal Values`.
- `Settings` → `Hexadecimal Addresses`.

1. **Assemble** (F3).
2. Clic sur **Reset** dans la fenêtre Instruction Counter (important : vider entre deux tests).
3. **Run** (F5) jusqu'à la fin.
4. Relever les chiffres.

Exécution. Dans `Instruction Counter` :

- Ligne *Instructions so far* → métrique principale.
- Décomposition R / I / S / B / U / J → utile pour distinguer calcul pur vs accès mémoire (les S-type = stores, les loads sont des I-type).

Mémoire — lecture directe dans RARS.

- **Text Segment** : scroller jusqu'à la dernière instruction de la fonction, noter l'adresse du label d'entrée et l'adresse juste après `ret`. Différence = taille du code en octets.
- **Data Segment** : idem pour `tab` et les constantes. On y voit le tableau avant/après le tri, utile pour vérifier.
- **Pile** : regarder `sp` dans la fenêtre *Registers* avant l'appel, puis mettre un breakpoint sur la 1re instruction de la fonction pour voir `sp` après le prologue. La différence = octets consommés sur la pile par appel. Pour un algo itératif comme le tri à bulles, elle ne grandit jamais au-delà.

4 Résultats

Les appels des fonctions communes sont considérés négligeables. Par exemple ***`print_tab`*** *est appelée de la même manière sur tous les algorithmes.*

Algorithme	Instr. tot.	R	I	S	B	U	J	Pile (o)	Récursif	Complexité
Tri à bulles	**3 034**	933	1 209	235	322	48	287	20	non	$O(n^2)$
Tri par sélection										$O(n^2)$
Tri par insertion										$O(n^2)$
Tri rapide									oui	$O(n \log n)$ moy.
Tri fusion									oui	$O(n \log n)$
Tri par tas	**3 375**	1 328	1 138	260	385	48	216	32	non	$O(n \log n)$

Table 1: Comparaison des algorithmes de tri sur un tableau de 20 entiers.

Type de comparaison	Formule	Ce que ça dit
Ratio entre algos	$\text{total}_A/\text{total}_B$	« A est 2,3× plus lent que B »
Intensité mémoire	$(I+S)/\text{total}$	part des instructions qui touchent la RAM
Vérification théorique	total/n^2 ou $\text{total}/(n\log_2 n)$	le coefficient doit rester à peu près constant si l'on refait le test avec un autre n

5 VRAC à travailler

5.1 Temps (= nombre d'instructions)

Comparaison directe : la colonne *Total instr.* suffit. Plus petit = plus rapide.

Trois lectures utiles :

La dernière ligne est la plus intéressante pour un rapport d'archi : elle valide (ou non) la complexité théorique. Pour la bulle avec $n = 20$, le coût « pur tri » $\approx 2698 -$ print instructions, divisé par $20^2 = 400 \to$ on obtient une constante $\sim X$. En refaisant avec $n = 50$, on doit retomber sur un ordre $\sim X$.

5.2 Mémoire

Là, un seul chiffre ne suffit pas. Il faut distinguer deux composantes.

Composant	Comment mesurer dans RARS
Taille du code de l'algo	adresse de `ret` − adresse du label d'entrée (en octets)
Données auxiliaires allouées	regarder `.data` du fichier : tables de travail, buffers...

a. Mémoire statique (code + données). Pour la bulle : juste du code, 0 octet de données auxiliaires (tri in-place). Pour le tri fusion : on allouera un tableau temporaire $\to +4n$ octets de données.

b. Mémoire dynamique (pile) — c'est ça qui diffère vraiment. La bonne métrique n'est pas un octet absolu, c'est la fonction de n :

Algo type	Pile consommée	Pour $n = 20$	Pour $n = 1000$
Itératif (bulle, insertion, sélection)	constante (1 cadre)	20 o	20 o
Récursif équilibré (fusion, tas)	cadre $\times \log_2(n)$	~ 5 cadres	~ 10 cadres
Récursif dégénéré (quicksort pire)	cadre $\times n$	20 cadres	1000 cadres !

Dans RARS on lit :

- **Taille d'un cadre** → dans le prologue : `addi sp, sp, -N` → N octets.
- **Profondeur max** → registre `sp` pendant l'exécution (breakpoint à l'intérieur de la fonction récursive), ou instrumentation `sp_min` si besoin.

Résumé : ce qu'il faut présenter dans le rapport. Deux tableaux, pas un.

Tableau 1 — Temps

Algo	Total instr.	Ratio vs plus rapide	Coefficient total/n^2 ou $/(n \log n)$

Tableau 2 — Mémoire

Algo	Code (o)	Données aux. (o)	Pile/cadre (o)	Profondeur pile	Pile max (o)

La colonne *Pile max* (= cadre × profondeur) est la métrique clé pour comparer l'empreinte mémoire dynamique entre algos.

Conclusion qualitative attendue. On devrait retrouver dans les chiffres :

- **Bulle** : lent ($O(n^2)$) mais mémoire minimale (20 o fixes).
- **Fusion** : rapide ($O(n \log n)$) mais paye en mémoire ($4n$ o de buffer + pile $\log_2 n$).
- **Quicksort** : rapide en moyenne, mais pile variable selon les données → risqué sur gros n.

Observation méthodologique : le piège du petit n. À $n = 20$, avec toutes les primitives factorisées (`swap(tab, i, j)` générique, appels via `jal`), les premiers chiffres relevés montrent un résultat *contre-intuitif* :

	Bulle	**Tas**
Instructions totales	3 034	**3 375**

Le tri par tas, pourtant $O(n \log n)$, est **plus coûteux** que la bulle à cette taille. L'explication est la même que partout en micro-benchmark algorithmique : à petit n, les **constantes cachées** (prologue/épilogue, appels de fonction, sauvegarde de registres) dominent l'avantage asymptotique. Le refactor vers un `swap` générique pénalise en plus les algos qui faisaient des échanges inlinés.

Mesures wall-clock à $n = 1\,000$ et $n = 10\,000$. Le passage à des tailles plus réalistes confirme l'inversion attendue. Mesures prises en ligne de commande avec `time rars test_tri_XXX.asm > /dev/null 2>&1` — stdout redirigé pour neutraliser le coût de rendu terminal des `print_tab`, seuls restent les `ecall` système et le tri proprement dit.

Algorithme	$n = 20$ **(instr.)**	$n = 1\,000$	$n = 10\,000$	**Complexité**
Tri à bulles	3 034	2,50 s	231,6 s	$O(n^2)$
Tri par sélection	—	2,57 s	223,9 s	$O(n^2)$
Tri par tas	3 375	**0,41 s**	**2,51 s**	$O(n \log n)$

Table 2: Temps wall-clock pour chaque algorithme aux trois tailles de test. Les colonnes $n = 1\,000$ et $n = 10\,000$ sont en secondes ; la baseline JVM (programme vide) est d'environ 187 ms.

Observations clés.

- À $n = 20$, les constantes dominent et le tri par tas est même *plus coûteux* que la bulle en nombre d'instructions (3 375 vs 3 034).
- À $n = 1\,000$, l'écart $\mathrm{O}(n^2)$ vs $\mathrm{O}(n \log n)$ devient visible : tas est ×6 plus rapide que bulle et sélection.

- À $n = 10\,000$, l'écart explose : tas est $\times 92$ plus rapide. Ratio théorique $n/\log_2 n \approx 750$; la différence s'explique par le coût des `ecall` de `print_tab` qui persiste même avec stdout redirigé, plus significatif pour les algos rapides.
- **Bulle et sélection sont empiriquement indiscernables** en wall-clock à $n = 1\,000$ et $n = 10\,000$, malgré un nombre d'échanges radicalement différent ($\sim n^2/2$ pour bulle, exactement $n - 1$ pour sélection). À cette échelle, le coût des **comparaisons** (identique pour les deux) domine celui des échanges.

Baseline et coût de `print_tab`. Le démarrage « à vide » de RARS (un `.asm` qui termine immédiatement) = **187 ms**. Tout ce qui dépasse cette baseline est imputable au programme de test.

Les `print_tab` contribuent à hauteur du nombre d'`ecall` émis, même lorsque stdout est redirigé vers `/dev/null` : chaque `ecall` passe par la machine virtuelle RARS, qui a un coût fixe non négligeable. À $n = 10\,000$, cela représente $4 \cdot 10^4$ `ecall` (deux appels à `print_tab`, chacun faisant n fois `print_int` + `print_string`), dont le coût devient dominant pour le tri par tas.

Conclusion : trois points de mesure complémentaires.

- $n = 20$ **(instructions)** : les constantes dominent ; la complexité asymptotique ne suffit pas à prédire l'ordre. Utile pour illustrer le *piège du petit n*.
- $n = 1\,000$ **(wall-clock)** : l'écart $\mathrm{O}(n^2)$ vs $\mathrm{O}(n \log n)$ devient visible sans attendre ; taille de dev confortable pour itérer.
- $n = 10\,000$ **(wall-clock)** : mise en évidence finale de la bascule ; valide expérimentalement la complexité annoncée.